

Prompt:

8/17/2026, 10:02:18 PM

lanjut Tahap Q

Response:

8/17/2026, 10:02:18 PM

Tahap Q — Event-Driven Backtesting & Market Simulator

Ya. **Tahap Q adalah titik yang sangat penting**, karena seluruh matematika A–P yang kita bangun sekarang harus ditempatkan ke dalam lingkungan yang bisa menjawab pertanyaan paling fundamental:

"Jika sistem ini benar-benar berada di pasar pada waktu t , dengan hanya informasi yang tersedia pada saat itu, apa yang sebenarnya akan terjadi?"

Tanpa ini, model kita mudah menghasilkan backtest yang terlihat luar biasa tetapi sebenarnya menggunakan *look-ahead bias*, mengabaikan spread, menganggap semua limit order pasti filled, atau menghitung stop-loss dengan informasi masa depan.

Jadi Tahap Q bukan sekadar "membuat backtester".

Kita akan membangun:

Event-Driven Market Simulator

yang menjadi **laboratorium virtual** untuk seluruh sistem.

Q.1. Paradigma utama

Backtester sederhana biasanya berpikir seperti:

$$P_t \rightarrow \text{Signal}_t \rightarrow \text{Return}_{t+1}.$$

Itu terlalu sederhana untuk sistem kita.

Kita ingin:

$$F_t \rightarrow \text{State}_t \rightarrow \text{Optimization}_t \rightarrow \text{Order}_t \rightarrow \text{Execution}_t \rightarrow \text{Position}_t \rightarrow \text{PnL}_t$$

dan kemudian:

$$\boxed{State_{t+1}}$$

bergantung pada apa yang benar-benar terjadi pada t .

Q.2. Event-driven, bukan candle-driven

Ini perbedaan fundamental.

Misalnya kita mempunyai candle:

$$O = 100, \quad H = 105, \quad L = 97, \quad C = 103.$$

Jika strategi memasang:

$$BUY \ LIMIT = 98$$

dan:

$$SL = 97,$$

kita **tidak bisa langsung mengatakan:**

"Low = 97, jadi limit order filled lalu stop-loss terkena."

Karena kita tidak tahu urutan intrabar.

Bisa saja:

$$100 \rightarrow 105 \rightarrow 98 \rightarrow 103 \rightarrow 97$$

atau:

$$100 \rightarrow 97 \rightarrow 98 \rightarrow 105 \rightarrow 103.$$

Konsekuensinya berbeda.

Q.3. Urutan kejadian adalah informasi

Misalkan:

$$TP = 105,$$

$$SL = 97.$$

Dalam satu candle:

$$H = 105, \quad L = 97.$$

Apakah TP atau SL terjadi terlebih dahulu?

Data OHLC saja tidak dapat menjawab.

Karena itu:

OHLC Backtest $\neq$ Market Simulation.

Q.4. Hierarki data

Untuk sistem kita, idealnya data dibagi menjadi:

Level 1 — Macro

DXY, Gold, SPX, Nasdaq, VIX, BTC, ETH, ...

Level 2 — Market

OHLCV.

Level 3 — Microstructure

Bid, Ask, Depth, Trades, Order Book.

Level 4 — Event

Order Submitted, Order Filled, Order Cancelled, Stop Triggered, TP Triggered.

Q.5. Informasi filtration

Ini bagian matematis yang sangat penting.

Misalkan:

$$\mathcal{F}_t$$

adalah seluruh informasi yang tersedia sampai waktu t .

Strategi harus memenuhi:

$$a_t \in \mathcal{F}_t.$$

Artinya keputusan pada waktu t **tidak boleh bergantung pada informasi dari $t + \Delta t$** .

Ini disebut *adapted strategy*.

Q.6. Larangan utama

Tidak boleh:

$$a_t = f(P_{t+1}).$$

Tidak boleh pula:

$$a_t = f(H_{t:t+1}, L_{t:t+1}).$$

Keputusan hanya boleh:

$$a_t = f(\mathcal{F}_t).$$

Q.7. Mengapa ini sangat penting untuk sistem kita?

Karena model kita memiliki:

$$\hat{\mu}_t,$$

$$\hat{\Sigma}_t,$$



$$P(Fill),$$

$$QUBO_t,$$

$$SL_t,$$

$$TP_t.$$

Semua harus dihitung berdasarkan:

$$\boxed{F_t}$$

dan bukan berdasarkan seluruh dataset.

Q.8. Rolling information set

Misalnya kita melakukan backtest pada:

$$2021 - 01 - 01 \rightarrow 2023 - 12 - 31.$$

Pada:

$$t = 2022 - 05 - 01,$$

model hanya boleh mengetahui:

$$2021 - 01 - 01 \rightarrow 2022 - 05 - 01.$$

Bukan:

$$2022 - 05 - 02 \rightarrow 2023 - 12 - 31.$$

Q.9. Training → validation → test

Dataset harus dibagi secara temporal.

Misalnya:

$$Train = 2021 - 2022$$

$$Validation = 2023\ Q1$$

$$Test = 2023\ Q2 - Q4$$

Tetapi untuk sistem yang adaptif, kita sebaiknya melangkah lebih jauh:

$$Walk\ Forward\ Validation.$$

Q.10. Walk-forward

Misalkan:

$$W_1 : \text{Train } [t_0, t_1] \rightarrow \text{Test } [t_1, t_2].$$

Kemudian:

$$W_2 : \text{Train } [t_0, t_2] \rightarrow \text{Test } [t_2, t_3].$$

Kemudian:

$$W_3 : \text{Train } [t_0, t_3] \rightarrow \text{Test } [t_3, t_4].$$

Sehingga:

Model selalu menghadapi masa depan yang belum dilihat.

Q.11. Purgings

Jika target prediction menggunakan horizon:

$$h$$

dan sample training berdekatan dengan validation, dapat terjadi overlap.

Misalnya:

$$X_t \rightarrow Y_{t+10}.$$

Kemudian validation dimulai pada:

$$t + 5.$$

Maka informasi target overlap.

Karena itu kita dapat menggunakan:

Purged Walk Forward.

Q.12. Embargo

Setelah training set berakhir, kita dapat memberikan gap:

$[t_{train,end}, t_{test,start}]$

untuk mengurangi leakage akibat dependensi temporal.

Q.13. Market simulator

Sekarang kita definisikan simulator:

M

yang menerima:

$$(state_t, event_t, action_t)$$

dan menghasilkan:

$$(state_{t+1}, event_{t+1}, reward_t).$$

Secara matematis:

$S_{t+1} = M(S_t, A_t, E_t).$

Q.14. State

State sistem:

$$S_t = \{M_t, F_t, O_t, P_t, R_t, C_t\}.$$

Dengan:

- M_t : market state,
 - F_t : factor state,
 - O_t : open orders,
 - P_t : positions,
 - R_t : risk state,
 - C_t : capital.
-

Q.15. Event

Event:

$$E_t \in E.$$

Misalnya:

$$E = \{MarketUpdate, Signal, OrderSubmit, OrderFill, OrderCancel, StopTrigger, TakeProfit, Timer\}.$$

Q.16. Event queue

Simulator mempunyai:

PriorityQueue

berdasarkan timestamp.

Misalnya:

```
09:30:00.000 MarketUpdate
09:30:00.005 Signal
09:30:00.006 OrderSubmit
09:30:00.120 Trade
09:30:00.121 PartialFill
09:30:02.000 Trade
09:30:02.001 Fill
09:31:15.000 Timer
```

Simulator memproses satu per satu.

Q.17. Event loop

Secara konseptual:

```
while event_queue not empty:

    event = next_event()

    update_market_state(event)

    process_open_orders()

    process_positions()

    if decision_event:
        generate_signal()

    if optimization_event:
        solve_QUBO()

    if execution_event:
        place_or_modify_orders()

    update_PnL()

    update_risk()

    record_state()
```

Ini adalah jantung backtester.

Q.18. Timestamp discipline

Setiap data harus memiliki:

$timestamp.$

Dan timestamp harus konsisten:

UTC

atau timezone yang jelas.

Tidak boleh mencampur:

$UTC + 0$

dengan:

$UTC + 7$

tanpa konversi.

Q.19. Clock model

Kita dapat menggunakan:

$$t_0 < t_1 < t_2 < \dots < t_N.$$

Setiap event:

$$e_i = (t_i, type_i, data_i).$$

Kemudian:

$$t_{i+1} \geq t_i.$$

Q.20. Order model

Setiap order kita representasikan sebagai:

$$O_i = (id, symbol, side, type, price, quantity, remaining, timestamp, status, SL, TP).$$

Misalnya:

$$O_i = (BTC, BUY, LIMIT, 100000, 0.001, \dots).$$

Q.21. Order status

$$\boxed{NEW \rightarrow OPEN \rightarrow PARTIAL \rightarrow FILLED}$$

atau:

$$OPEN \rightarrow CANCELLED.$$

atau:

$$PARTIAL \rightarrow CANCELLED.$$

Q.22. Market order simulation

Market buy:

$$P_{exec} = VWAP_{ask}(q).$$

Market sell:

$$P_{exec} = VWAP_{bid}(q).$$

Jika hanya OHLCV tersedia, kita perlu menggunakan model slippage.

Misalnya:

$$P_{exec} = P_t + \epsilon(q, \sigma, D).$$

Q.23. Limit order simulation

Jika:

$$P_{ask} \leq P_{limit}$$

untuk buy, order berpotensi filled.

Tetapi **tidak otomatis filled**.

Kita harus mempertimbangkan:

$$Q_{queue}.$$

Q.24. Queue simulator

Misalkan:

$$Q_t = 500.$$

Order kita:

$$q = 100.$$

Kemudian market sells consume:

$$200.$$

Maka:

$$Q_{remaining} = 300.$$

Order kita belum filled.

Jika kemudian:

$$400$$

unit dikonsumsi:

$$300$$

menghabiskan queue depan.

Maka:

$$100$$

dapat mengisi order kita.

Q.25. Partial fill

Jika hanya:

$$70$$

unit yang tersedia setelah queue habis:

$$q_{filled} = 70.$$

Remaining:

$$q_{remaining} = 30.$$

Simulator harus mempertahankan state tersebut.

Q.26. Fill probability model

Jika order book tidak tersedia secara historis, kita dapat membuat:

$$p_F = f(\text{spread}, \text{depth}, \text{imbalance}, \text{queue}, \text{volatility}, \text{flow}, \text{distance}).$$

Misalnya logistic model:

$$p_F = \sigma(\beta_0 + \beta^T x)$$

dengan:

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Q.27. Tetapi ada masalah besar

Kita **tidak boleh** melatih:

$$p_F$$

dengan data masa depan lalu menggunakannya pada masa lalu.

Jadi:

$$\beta$$

harus diperoleh dari training window.

Kemudian digunakan pada test window.

Q.28. Stop-loss simulation

Misalkan posisi long:

$$P_{entry} = 100.$$

SL:

$$95.$$

Jika market tick menyentuh:

$$95,$$

stop triggered.

Tetapi execution mungkin:

$$95.1$$

atau:

$$94.8$$

karena slippage.

Maka:

$$P_{exit} \neq P_{SL}.$$

Q.29. Gap risk

Misalkan:

$$SL = 95.$$

Tetapi harga berikutnya:

$$90.$$

Maka:

$$P_{exit} \approx 90,$$

bukan:

$$95.$$

Ini penting untuk saham.

Untuk crypto yang berjalan 24/7, gap masih dapat terjadi secara efektif melalui lonjakan harga, meskipun mekanisme penutupan pasar berbeda.

Q.30. TP juga bukan guaranteed price

Jika:

$$TP = 105$$

dan market melompat:

$$103 \rightarrow 108,$$

execution dapat:

$$P_{exit} > 105.$$

Sebaliknya jika liquidity buruk:

$$P_{exit}$$

bisa lebih buruk.

Q.31. PnL

Untuk long:

$$PnL_t = q(P_t - P_{entry}) - C_t.$$

Untuk short:

$$PnL_t = q(P_{entry} - P_t) - C_t.$$

Dengan:

$$C_t = fees + slippage + impact.$$

Q.32. Realized dan unrealized PnL

$$PnL_t = PnL_t^{realized} + PnL_t^{unrealized}.$$

Realized:

$$PnL^{realized}$$

hanya berasal dari posisi yang sudah ditutup.

Q.33. Equity curve

Capital:

$$C_0.$$

Maka:

$$C_t = C_0 + \sum_{\tau \leq t} P n L_{\tau}.$$

Equity curve:

$$E(t) = C_t.$$

Ini yang akhirnya kita gunakan untuk evaluasi strategi.

Q.34. Drawdown

High-water mark:

$$H_t = \max_{\tau \leq t} E_{\tau}.$$

Drawdown:

$$DD_t = \frac{H_t - E_t}{H_t}.$$

Maximum drawdown:

$$MDD = \max_t DD_t.$$

Q.35. Tetapi Sharpe Ratio saja tidak cukup

Kita akan mengukur:

$$Sharpe = \frac{E[R_p - R_f]}{\sigma(R_p)}.$$

Tetapi juga:

Sortino,

Calmar,

MDD,

Profit Factor,

Win Rate,

Expectancy,

Tail Risk.

Q.36. Trade expectancy

Jika:

$$p_w = P(\text{win}),$$

$$G = E[\text{gain}],$$

$$L = E[\text{loss}],$$

maka:

$$E[\Pi] = p_w G - (1 - p_w)L.$$

Namun untuk sistem kita, expectancy harus sudah memasukkan:

fees,

slippage,

execution.

Q.37. Contoh penting

Strategi A:

$$WinRate = 60\%.$$

Strategi B:

$$WinRate = 40\%.$$

Belum tentu A lebih baik.

Misalnya:

A

$$Gain = 1$$

$$Loss = 2.$$

Expectancy:

$$0.6(1) - 0.4(2) = -0.2.$$

B

$$Gain = 3$$

$$Loss = 1.$$

$$0.4(3) - 0.6(1) = 0.6.$$

Jadi:

$WinRate \nexists Strategy\ Quality.$

Q.38. Risk-adjusted performance

Karena kita menetapkan:

$$R_{trade} \leq \$1,$$

kita juga dapat mengukur:

Return/Risk.

Misalnya:

$$\frac{E[PnL]}{E[Risk]}.$$

Ini lebih relevan terhadap filosofi sistem kita.

Q.39. Transaction Cost Analysis

Tahap Q juga harus memecah PnL:

Gross Alpha

menjadi:

$$Gross\ Alpha - Spread - Slippage - Fees - Impact - Opportunity\ Cost.$$

Sehingga:

Net Alpha

dapat dihitung.

Q.40. Ini akan sangat penting

Bayangkan:

$$Gross\ Return = 20\%.$$

Tetapi:

$$Transaction\ Cost = 18\%.$$

Maka:

$$Net = 2\%.$$

Model tersebut secara praktis hampir tidak berguna.

Q.41. Sensitivity analysis

Kita tidak boleh hanya menjalankan satu konfigurasi.

Misalnya:

$$Spread \times \{0.5, 1, 2\}$$

$$Slippage \times \{0.5, 1, 2\}$$

$$Fee \times \{0.5, 1, 2\}.$$

Kemudian lihat apakah strategi tetap profitable.

Q.42. Stress testing

Market regime:

$$R = \{Normal, HighVol, Crash, LowLiquidity, Trend, MeanReversion\}.$$

Backtester harus mengukur:

$$Performance(R_i).$$

Q.43. Contoh

Jika:

$$Sharpe_{normal} = 2.1$$

tetapi:

$$Sharpe_{crash} = -4.0,$$

kita harus memahami mengapa.

Bukan sekadar mengatakan:

"Sharpe total = 1.7."

Q.44. Monte Carlo trade reshuffling

Setelah memperoleh sequence:

$$\Pi_1, \Pi_2, \dots, \Pi_N,$$

kita dapat melakukan bootstrap.

Misalnya:

$$10,000$$

simulasi urutan trade.

Kemudian hitung:

$$MDD^{(i)}.$$

Sehingga diperoleh distribusi:

$$P(MDD > x).$$

Q.45. Ini membantu menjawab:

"Apakah drawdown terbesar yang saya lihat memang representatif?"

Misalnya backtest menunjukkan:

$$MDD = 12\%.$$

Tetapi Monte Carlo menunjukkan:

$$95\% \text{ quantile} = 24\%.$$

Maka strategi sebenarnya jauh lebih berisiko daripada yang tampak dari satu historical path.

Q.46. Parameter uncertainty

Kita juga harus mempertimbangkan:

$$\hat{\theta}$$

bukan:

$$\theta.$$

Misalnya:

$$\hat{\beta}, \hat{\Sigma}, \hat{\mu}, \hat{\lambda}.$$

Semua memiliki uncertainty.

Maka:

$$\boxed{Model\ Risk}$$

harus masuk dalam evaluasi.

Q.47. Optimizer uncertainty

Ini bahkan lebih relevan untuk quantum.

Misalkan:

$$QUBO^*$$

menghasilkan solusi:

$$x^*.$$

Tetapi QAOA hanya memperoleh:

$$\tilde{x}.$$

Kita harus mengukur:

$$\text{Gap} = E(x^*) - E(\tilde{x}).$$

Q.48. Quantum trading backtest

Ini akan menjadi salah satu bagian paling menarik dari proyek kita.

Untuk setiap problem:

$$Q_t$$

kita dapat menjalankan:

Exact

$$x_t^{\text{exact}}.$$

Classical heuristic

$$x_t^{\text{classical}}.$$

QAOA

$$x_t^{\text{QAOA}}.$$

Kemudian:

$$PnL^{\text{exact}},$$

$$PnL^{\text{classical}},$$

$$PnL^{\text{QAOA}}.$$

Q.49. Tetapi jangan langsung membandingkan PnL

Misalnya QAOA menghasilkan PnL:

10%.

Classical:

12%.

Belum tentu QAOA buruk.

Kita harus mengetahui:

Problem Size,

Runtime,

Solution Quality,

Constraint Violation.

Q.50. Approximation ratio

Untuk minimization:

$$\rho = \frac{E_{QAOA}}{E_{optimal}}$$

dengan definisi yang harus disesuaikan dengan konvensi objective.

Untuk maximization:

$$\rho = \frac{E_{QAOA}}{E_{optimal}}.$$

Jika:

$$\rho \rightarrow 1,$$

solusi semakin dekat optimum.

Q.51. Yang lebih penting: economic degradation

Kita dapat mendefinisikan:

$$\Delta PnL = PnL_{optimal} - PnL_{QAOA}.$$

Kemudian:

$$\text{Economic Cost of Approximation}$$

adalah:

$$EC_{approx} = \Delta PnL.$$

Jadi kita tidak hanya mengatakan:

"QAOA mempunyai approximation ratio 95%."

Tetapi:

"Approximation error 5% tersebut menyebabkan penurunan expected PnL sebesar X."

Ini jauh lebih bermakna secara finansial.

Q.52. Event-driven backtest architecture

Saya menyarankan struktur:

```
TradeProject/
|
├─ data/
|   ├── raw/
|   ├── processed/
|   └─ features/
|
├─ market/
|   ├── market_state.py
|   ├── order_book.py
|   └─ factor_state.py
|
├─ strategy/
|   ├── alpha.py
|   ├── signal.py
|   └─ regime.py
|
├─ optimization/
|   ├── portfolio_qubo.py
|   ├── entry_qubo.py
|   ├── management_qubo.py
|   └─ execution_qubo.py
|
├─ execution/
|   ├── order.py
|   ├── fill_model.py
|   ├── slippage.py
|   └─ impact.py
|
├─ portfolio/
|   ├── position.py
|   ├── pnl.py
|   └─ risk.py
|
├─ backtest/
|   ├── engine.py
|   ├── event.py
|   ├── clock.py
|   └─ simulator.py
|
├─ evaluation/
|   └─ metrics.py
```

```
| |─ drawdown.py
| |─ monte_carlo.py
| |─ stress_test.py
|
|─ experiments/
```

Q.53. Satu prinsip arsitektur yang sangat penting

Strategy tidak boleh mengetahui masa depan.

Jadi jangan membuat:

```
strategy(data)
```

dengan:

```
data = entire_dataset
```

lalu strategy mengambil apa saja.

Lebih aman:

```
strategy.on_event(event, state)
```

dan:

```
state
```

hanya berisi informasi sampai:

t .

Q.54. Data leakage audit

Kita harus memiliki pemeriksaan:

$timestamp(data) \leq timestamp(decision).$

Untuk setiap feature:

$$X_{i,t}$$

harus memenuhi:

$$X_{i,t} \in F_t.$$

Q.55. Contoh bug yang sangat berbahaya

Misalkan kita menghitung:

20-day moving average

menggunakan:

```
rolling(20).mean()
```

itu relatif aman.

Tetapi jika kita menggunakan:

```
center=True
```

maka feature pada t dapat menggunakan:

$$P_{t+1}, P_{t+2}, \dots$$

dan backtest menjadi invalid.

Q.56. Bug lain: normalization

Jika kita melakukan:

$$z_t = \frac{x_t - \mu_{all}}{\sigma_{all}}$$

dengan:

$$\mu_{all}$$



dihitung menggunakan seluruh dataset, maka future information bocor.

Harus:

$$z_t = \frac{x_t - \mu_t^{past}}{\sigma_t^{past}}.$$

Q.57. Bug lain: covariance

Dalam portfolio optimizer kita menggunakan:

$$\Sigma_t.$$

Jangan menghitung:

$$\Sigma_{2021-2023}$$

untuk membuat keputusan pada:

2021.

Harus:

$$\Sigma_t = \text{Cov}(r_{t-L:t}).$$

Q.58. Bug lain: universe selection

Misalnya memilih saham berdasarkan:

"saham yang sekarang masih aktif dan performanya bagus."

Kemudian digunakan untuk backtest 2015.

Ini:

Survivorship Bias.

Universe harus direkonstruksi berdasarkan informasi yang tersedia pada saat itu.

Q.59. Bug lain: delisting

Jika saham hilang dari dataset karena bangkrut, lalu kita hanya menyimpan saham yang masih ada, hasil backtest terlalu optimistis.

Karena itu dataset historical harus mempertahankan:

delisted assets.

Q.60. Bug lain: corporate actions

Untuk saham:

split, dividend, rights, merger

harus diperhatikan.

Jika tidak, return dapat salah.

Q.61. Crypto memiliki masalah yang berbeda

Crypto:

- 24/7,
- exchange fragmentation,
- funding rate,
- liquidation,
- perpetual futures,
- maker/taker fee,
- spread yang berubah cepat.

Jadi simulator crypto perlu:

Funding Model.

Q.62. Jika menggunakan perpetual futures

PnL tidak hanya:

$price PnL$.

Tetapi:

$$PnL = PricePnL + FundingPnL - Fees - Slippage.$$

Funding:

$$FundingPnL = qP_t f_t$$

dengan konvensi tanda yang sesuai dengan posisi long/short dan funding rate.

Q.63. Untuk saham internasional

Kita perlu:

FX .

Misalnya saham Jepang:

JPY .

Tetapi account base:

USD .

Maka:

$$PnL_{USD} = PnL_{JPY} \times FX_{JPY/USD}.$$

Ini membuat faktor:

DX

dan FX exposure menjadi bagian dari risk model.

Q.64. Gold dan USD dominance

Karena kita sebelumnya ingin sistem lintas pasar, state global dapat:

$$F_t = [r_{Gold}, r_{DXY}, r_{SPX}, r_{NQ}, r_{BTC}, r_{ETH}, r_{VIX}, \dots].$$

Kemudian:

$$r_i = \alpha_i + \beta_i^T F_t + \epsilon_i.$$

Q.65. Event-driven factor update

Jika:

$$DXY$$

berubah pada:

$$t,$$

maka factor state langsung berubah:

$$F_t \rightarrow F_{t+1}.$$

Kemudian:

$$\hat{\mu}_{asset}$$

dihitung ulang.

Dengan demikian:

$$\boxed{\text{Macro information} \rightarrow \text{Asset forecast} \rightarrow \text{Optimization}}$$

secara dinamis.

Q.66. Re-optimization

Kita tidak perlu menjalankan QUBO pada setiap tick.

Kita dapat menetapkan:

$$\Delta t_{opt}.$$

Misalnya:

$$\Delta t_{opt} = 1h.$$

Tetapi execution dapat berjalan pada resolusi lebih tinggi.

Jadi:

$$\text{Forecast/Optimization : } 1h$$

sementara:

$$\text{Execution : tick/second/minute}$$

tergantung data.

Q.67. Ini menghasilkan multi-timescale architecture

Misalnya:

$$\text{Macro : } 1D$$

$$\text{Factor : } 1H$$

$$\text{Alpha : } 5M$$

$$\text{Execution : } 1S$$

atau bahkan:

$$\text{Tick.}$$

Q.68. Ini sangat cocok dengan konsep sebelumnya

Kita sebenarnya memiliki:

Slow State

dan:

Fast State.

Slow:

DEX, Gold, regime, covariance.

Fast:

Orderbook, imbalance, spread, flow.

Kemudian:

$Decision = f(SlowState, FastState).$

Q.69. Akhirnya kita mendapatkan hierarchical control

Secara konseptual:

Strategic Layer

menentukan:

What.

Tactical Layer

menentukan:

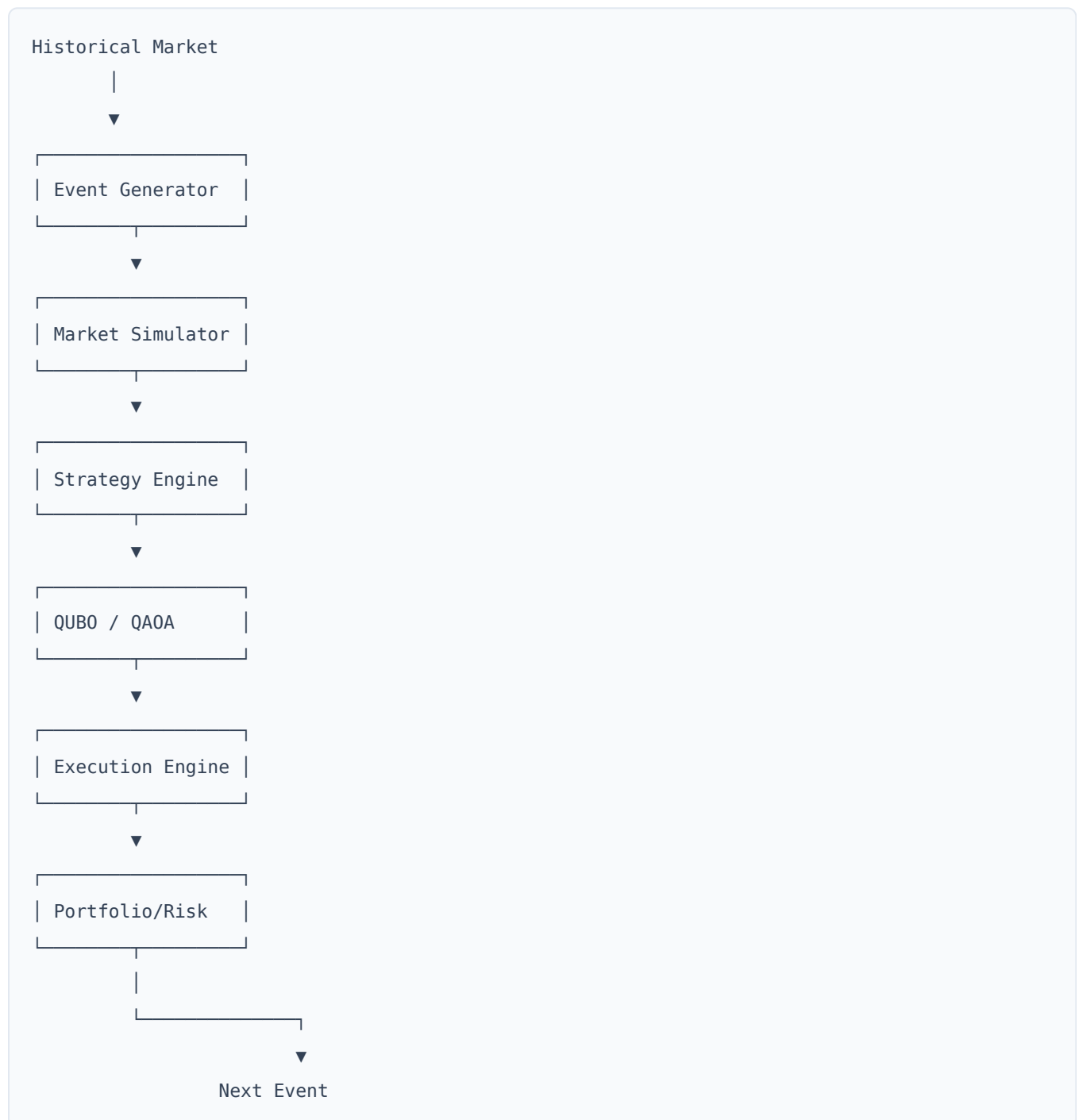
When.

menentukan:

How.

Q.70. Dan semuanya diuji oleh simulator

Jadi:



Q.71. Kriteria bahwa backtester kita sudah "benar"

Saya tidak ingin kita mengatakan:

"Backtester selesai karena grafik equity curve sudah muncul."

Backtester baru layak dipercaya jika kita bisa membuktikan:

1. No look-ahead

$$a_t \in \mathcal{F}_t.$$

2. No survivorship bias

Universe historical valid.

3. Realistic execution

$$P_{exec} \subseteq P_{close}.$$

4. Realistic limit fill

$$P(\text{Fill}) < 1.$$

5. Transaction costs

$$Fee + Spread + Slippage + Impact.$$

6. Partial fills

Didukung.

7. Dynamic SL/TP

Didukung.

8. Multi-asset capital constraint

Didukung.

9. Walk-forward

Didukung.

10. Monte Carlo/stress test

Didukung.

Q.72. Unit test yang harus kita buat

Sebelum menjalankan strategi besar, kita buat market kecil yang kita **tahu jawabannya**.

Misalnya:

$$P_0 = 100,$$

kemudian:

$$100 \rightarrow 101 \rightarrow 102.$$

Jika:

$$BUY = 100,$$

$$SELL = 102,$$

maka:

$$PnL = 2q.$$

Simulator harus menghasilkan tepat itu.

Kemudian tambahkan:

$$fee.$$

Kemudian:

$$slippage.$$

Kemudian:

$$partialfill.$$



Dengan demikian setiap komponen dapat diverifikasi.

Q.73. Invariant risk

Simulator harus selalu memeriksa:

$$R_{open} \leq R_{max}.$$

Jika:

$$R_{open} > \$1$$

untuk sebuah trade, sistem harus menghasilkan error atau reject.

Bukan diam-diam melanjutkan.

Q.74. Capital invariant

Jika capital:

$$C_t$$

maka:

$$Margin_t \leq C_t$$

atau sesuai leverage/margin rules.

Tidak boleh:

$$Position > AvailableCapital$$

kecuali leverage memang dimodelkan.

Q.75. Position invariant

Misalnya strategi melarang short.

Maka:

$$q_t \geq 0.$$

Jika futures:

$$q_t \in \mathbb{R}.$$

Semua constraint harus eksplisit.

Q.76. Audit trail

Setiap keputusan harus dapat dijelaskan.

Misalnya:

2026-08-17 10:00

Signal:

LONG SOL

Expected return:

0.31%

Risk:

\$0.73

QUBO solution:

BUY = 1

Execution:

LIMIT @ 182.40

Fill probability:

0.71

Queue:

1,243 SOL

Expected slippage:

0.004%

Decision:

PLACE LIMIT

Ini akan sangat berguna ketika kita melakukan debugging.

Q.77. Bahkan QAOA harus diaudit

Misalnya:

QA0A

n_qubits = 12

p = 2

shots = 4096

best bitstring:

101001101010

objective:

-17.42

exact optimum:

-17.91

approximation ratio:

0.973

Kemudian:

economic impact

dari solusi tersebut dicatat.

Q.78. Backtest report

Setiap eksperimen sebaiknya menghasilkan:

Experiment ID

Data Version

Feature Version

Model Version

QUBO Version

Optimizer Version

Transaction Cost Model

Random Seed

Start Date

End Date

Sehingga eksperimen:

$$E_1$$

dapat direproduksi.

Q.79. Ini penting untuk quantum

Karena QAOA stochastic.

Misalnya:

$$seed = 1$$

menghasilkan:

$$PnL = 14\%.$$

Sedangkan:

$$seed = 2$$

menghasilkan:

$$PnL = 8\%.$$

Kita tidak boleh memilih seed yang paling bagus.

Harus:

$$\boxed{E_{\text{seed}}[PnL]}$$

dan:

$$Var_{\text{seed}}(PnL).$$

Q.80. QAOA sebagai stochastic optimizer

Secara konseptual:

$$x^* = \text{QAOA}(Q_t; \theta, \xi)$$

dengan:

$$\xi$$

adalah stochastic measurement noise.

Maka:

$$P(x)$$

bukan deterministic.

Backtester harus mampu menangani hal tersebut.

Q.81. Ini membawa kita ke metrik yang sangat menarik

Bukan hanya:

$$PnL.$$

Tetapi:

$$PnL \text{ per QAOA evaluation.}$$

atau:

$$Economic \text{ Value/Computational Cost.}$$

Misalnya:

$$EV_{quantum} = \frac{E[PnL_{QAOA}] - E[PnL_{classical}]}{Runtime_{QAOA}}.$$

Ini jauh lebih dekat dengan pertanyaan:

Apakah optimasi quantum benar-benar memberikan nilai ekonomi?

Q.82. Tahap Q mengubah proyek kita

Sebelum Q:

"Kita mempunyai model trading yang kompleks."

Setelah Q:

"Kita mempunyai eksperimen yang dapat diverifikasi terhadap market timeline."

Itu perbedaan besar.

Q.83. Hubungan seluruh tahapan

Sekarang pipeline kita secara konseptual:

$$A \rightarrow B \rightarrow C \rightarrow \cdots \rightarrow P \rightarrow Q$$

dan Q bertindak sebagai **validation environment**.

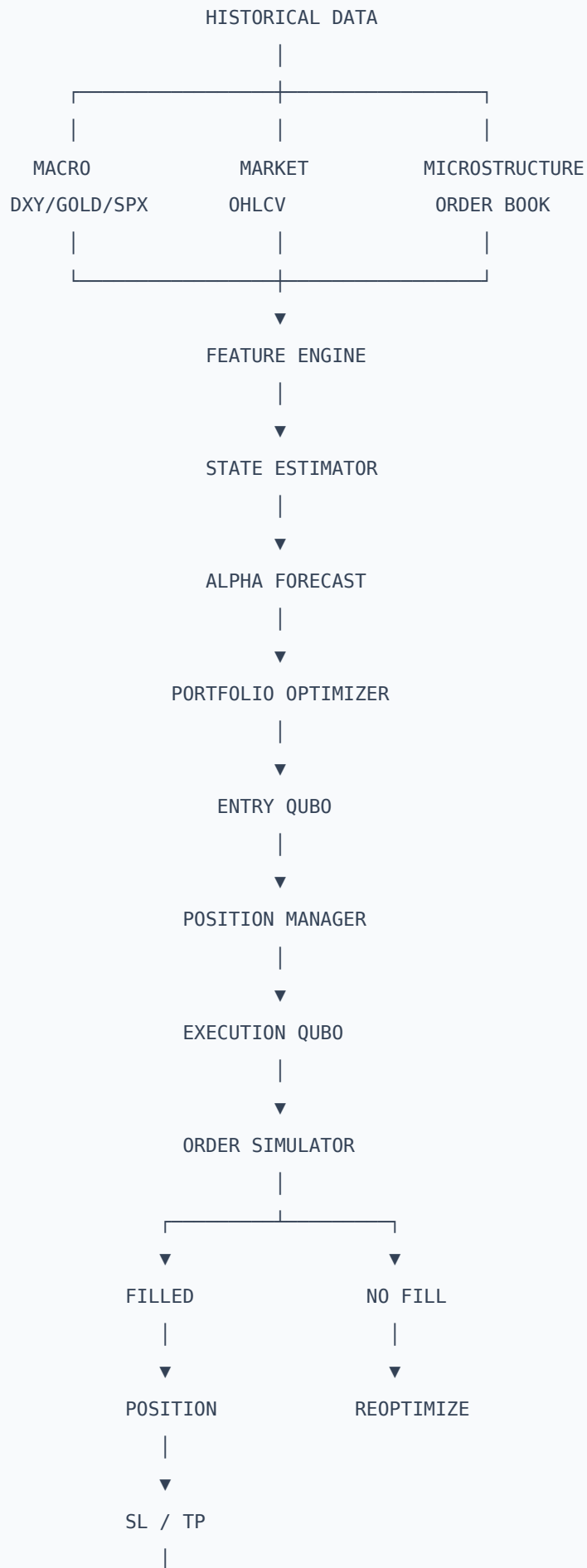
Lebih tepatnya:

$$Q \supset A-P$$

dalam arti simulator menjalankan seluruh komponen tersebut dalam waktu historis.

Q.84. Arsitektur final sementara

Saya akan menggambarkannya seperti ini:





Q.85. Dan sekarang kita sampai pada satu titik yang sangat menarik

Kita sebenarnya telah mengubah masalah:

"Buat trading bot."

menjadi:

Sequential Stochastic Constrained Optimization Problem

dengan state:

S_t ,

action:

A_t ,

constraint:

$g(S_t, A_t) \leq 0$,

objective:

$$J = E \left[\sum_t \gamma^t R(S_t, A_t) \right].$$

Tetapi **kita belum perlu menyebutnya reinforcement learning**.

Kita dapat menyelesaikan setiap keputusan menggunakan optimizer yang sesuai.

Q.86. Formulasi matematis keseluruhan

Secara abstrak:

$$S_{t+1} = F(S_t, A_t, \epsilon_t)$$

dengan:

$$A_t = \arg \max_{a \in A(S_t)} U(S_t, a).$$

Reward:

$$R_t = P n L_t - C_t - \lambda Risk_t.$$

Kemudian objective:

$$\max_{\pi} E_{\pi} \left[\sum_{t=0}^T \gamma^t R_t \right].$$

Ini adalah fondasi matematis yang nantinya memungkinkan kita, **jika memang diperlukan**, mengubah beberapa modul menjadi MDP/RL.

Q.87. Tetapi untuk sekarang saya akan mempertahankan prinsip

$$\text{Don't use RL just because we can.}$$

Kita ingin setiap komponen memiliki interpretasi finansial dan matematis:



Dengan demikian apabila sistem gagal, kita dapat mengetahui:

di mana ia gagal.

Q.88. Tahap Q selesai jika kita mampu melakukan eksperimen ini

Misalnya:

2023 - 01 - 01 → 2023 - 12 - 31

dan simulator menghasilkan:

Capital	
Initial	\$1,000
Final	\$1,137
Trades	428
Win rate	47.4%
Profit factor	1.31
Sharpe	1.42
Sortino	1.96
Max DD	8.7%
Avg risk	\$0.73
Max risk	\$1.00
Avg slippage	0.031%
Avg fee	0.040%
Limit fills	62.1%
Partial fills	14.7%

Tetapi yang jauh lebih penting:

setiap angka tersebut harus dapat ditelusuri kembali ke event dan keputusan yang menghasilkan angka itu.

Q.89. Deliverable Tahap Q

Jadi implementasi nyata Tahap Q nantinya harus menghasilkan minimal:

Engine

EventDrivenBacktester

Market simulator

MarketSimulator

Order simulator

OrderBookSimulator

Fill model

FillModel

Cost model

TransactionCostModel

Portfolio

PortfolioLedger

Risk

RiskEngine

Evaluation

PerformanceAnalyzer

Validation

WalkForwardEngine

Robustness

MonteCarloEngine

Q.90. Dan satu aturan emas

Jangan optimasi QAOA terhadap historical PnL secara langsung terlebih dahulu.

Pertama:

Past Data → Valid Simulator



kemudian:

Simulator → Baseline Strategy

kemudian:

Baseline → Classical Optimization

baru:

Classical → QAOA

Dengan demikian jika QAOA menghasilkan hasil buruk, kita bisa mengetahui apakah masalahnya berasal dari:

- data,
- signal,
- formulasi QUBO,
- optimizer,
- execution,
- atau memang QAOA.

Posisi kita sekarang

Tahap Q memberi kita "**physics engine**" untuk eksperimen trading.

Secara keseluruhan:

Market → State → Forecast → Optimization → Execution → Market Response

dan loop tersebut berjalan:

$$t = 0, 1, 2, \dots, T.$$

Itulah yang membuat sistem kita berbeda dari sekadar script trading yang membaca candle lalu membeli/jual.

Tahap berikutnya, Tahap R, menurut saya sebaiknya menjadi *Robustness, Statistical Validation & Anti-Overfitting*. Setelah kita memiliki simulator event-driven, kita perlu menjawab pertanyaan yang jauh lebih keras: *apakah edge yang kita lihat benar-benar ada, atau hanya artefak dari parameter tuning?* Di sana kita bisa masuk ke **Deflated Sharpe Ratio, Probability of Backtest Overfitting, White's Reality**

Check, stationary/block bootstrap, purged walk-forward, regime-conditioned performance, multiple-hypothesis correction, dan akhirnya menentukan apakah suatu hasil QAOA/classical optimizer layak disebut *genuine statistical edge*.